

Experiment-1

Aim: Lab on Web Services.

Description:

- Web services enable **interoperability between software systems** by allowing applications to exchange data over the internet using standardized communication protocols.
- They operate independently of user interfaces and are designed specifically for **automated system-level communication**.
- Web services are a core component of **distributed and service-oriented architectures**.
- One application → to send a request
Another application → to process it and send back a response

Categories of Web Services

- **REST-based Services**
- **SOAP-driven Services**
- **GraphQL-based Services**

1)REST API (Node.js with Express Framework)

- REST APIs follow a **resource-oriented design**, where each resource is accessed through a unique URL.
- They rely on **stateless communication**, improving scalability and performance.
- Standard HTTP actions are used to manage resources efficiently.

Used For

- ✓ Mobile apps
- ✓ Web apps
- ✓ CRUD apps
- ✓ Public APIs

Real-world Examples

- GitHub API
- Spotify API
- Instagram API
- Firebase REST API

1. Creating a Simple REST Web Service

First create project Folder by graphically or command prompt: open command prompt and run following commands

```
C:\Users\ADMIN>d:
D:\>mkdir cc_lab\week1
D:\>cd cc_lab\week1
D:\cc_lab\week1>
```

STEP 1: Install Python Requirements

1. Make sure Python is installed

python --version

```
D:\cc_lab\week1>python --version
Python 3.10.11
```

2. Run these commands:

pip install fastapi uvicorn

This installs:

FastAPI → to create the API

Uvicorn → to run the API server

```
D:\cc_lab\week1>pip install fastapi uvicorn
Requirement already satisfied: fastapi in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (0.122.0)
Requirement already satisfied: uvicorn in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (0.38.0)
Requirement already satisfied: starlette<0.51.0,>=0.40.0 in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (0.50.0)
Requirement already satisfied: pydantic<1.8.1,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4 in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (2.12.5)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (4.15.0)
Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (0.0.4)
```

STEP 3: Create main.py File

Write the following code inside **main.py** file and save it **inside week1 folder**

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def home():
    return {"message": "Welcome to My First Web Service"}
@app.get("/add/{a}/{b}")
def add(a: int, b: int):
    result = a + b
    return {"A": a, "B": b, "Result": result}
```

STEP 4: Run the Web Service

Go to your project folder in CMD/Terminal and run the following command

uvicorn main:app --reload

```
D:\cc_lab\week1>uvicorn main:app --reload
[32mINFO[0m: Will watch for changes in these directories: ['D:\\cc_lab\\week1']
[32mINFO[0m: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
[32mINFO[0m: Started reloader process [36m[1m4928[0m] using [36m[1mStatReload[0m
[32mINFO[0m: Started server process [36m[1m7664[0m]
[32mINFO[0m: Waiting for application startup.
[32mINFO[0m: Application startup complete.
```

If everything is correct, you will see:

Uvicorn running on <http://127.0.0.1:8000>

```
[32mINFO[0m: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

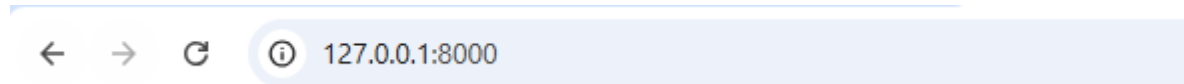
(2 line in the above output)

STEP 5: Test the API in Browser

Home Endpoint:

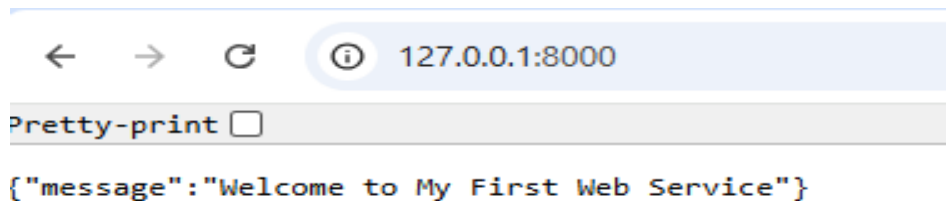
Open browser and type:

<http://127.0.0.1:8000/>



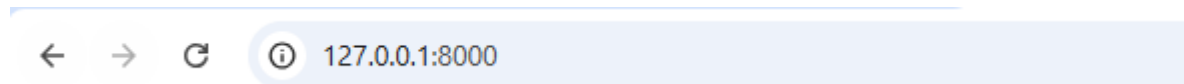
You will see:

```
{
  "message": "Welcome to My First Web Service"
}
```



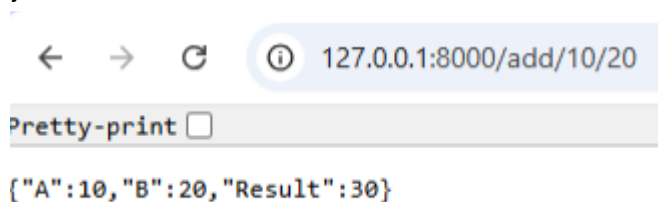
Add Two Numbers Endpoint:

<http://127.0.0.1:8000/add/10/20>



Output:

```
{
  "A": 10,
  "B": 20,
  "Result": 30
}
```



2)SOAP Web Service

- **SOAP (Simple Object Access Protocol)** is a protocol-driven web service standard that uses **XML** for structured and secure message exchange.
- It enforces strict messaging rules, making it highly reliable for **mission-critical applications**.
- SOAP services are platform-independent and support advanced features like **security, transactions, and error handling**.
- Due to their robustness, SOAP services are widely preferred in **regulated and enterprise environments**.
- SOAP ensures high data integrity during complex service communications.

Applications of SOAP Web Services

- Banking and payment processing systems
- Enterprise-level business applications
- Government and public sector platforms
- Financial and transaction-based services

Industry Use Cases

- PayPal Classic API
- Airline reservation and booking systems
- Insurance policy management services

Creating a Simple SOAP Web Service (Python + Spyne)

- Spyne is a lightweight Python framework used to develop SOAP-based web services.
- This setup is compatible specifically with **Python version 3.9.x**.

Step 1: Install Spyne

- Open **Command Prompt / Terminal** in the project directory
- Execute the following command to install Spyne:

pip install spyne

Note: spyne is a python library for building SOAP based web service in python.

2. Also install lxml (needed for validation)

pip install lxml

3. Create a **soap_service.py** file and write the following code and save it.

```
from spyne import Application, rpc, ServiceBase, Integer
```

```
from spyne.protocol.soap import Soap11
```

```
from spyne.server.wsgi import WsgiApplication
```

```
from wsgiref.simple_server import make_server
```

```
class CalculatorService(ServiceBase):
```

```
    @rpc(Integer, Integer, _returns=Integer)
```

```
    def add(ctx, a, b):
```

```
        return a + b
```

```
    application = Application(
```

```
[CalculatorService],
tns='calculator.soap.service',
in_protocol=Soap11(validator='lxml'),
out_protocol=Soap11()
)
wsgi_app = WsgiApplication(application)
if __name__ == "__main__":
server = make_server('127.0.0.1', 8001, wsgi_app)
print("SOAP service running on http://127.0.0.1:8001")
server.serve_forever()
```

STEP 2: Run the SOAP Server

```
D:\cc_lab\week1> python soap_service.py
SOAP service running on http://127.0.0.1:8001
127.0.0.1 - - [01/Dec/2025 11:12:05] "GET / HTTP/1.1" 405 389
127.0.0.1 - - [01/Dec/2025 11:12:05] "GET /favicon.ico HTTP/1.1" 405 389
```

You will see:

SOAP service running on <http://127.0.0.1:8001>

STEP 3: Open the WSDL

Open your browser:

<http://127.0.0.1:8001/?wsdl>

You will see the XML WSDL doc



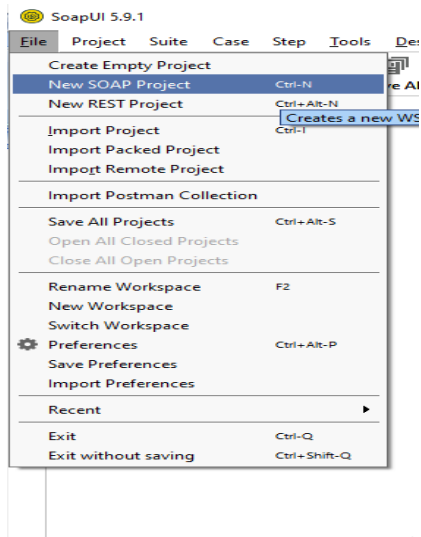
The screenshot shows a web browser window with the address bar displaying 127.0.0.1:8001/?wsdl. Below the address bar, a message states: "This XML file does not appear to have any style information associated with it. The document tree is shown below:". The XML content is displayed in a tree view, showing the WSDL structure for the 'calculator.soap.service' application. Key elements include the 'add' operation, its input 'a' and 'b' (both integers), and its output 'addResult' (integer). The service is located at 'http://127.0.0.1:8001/'.

STEP 4: Test the SOAP Service Using SoapUI

After installation (<https://www.soapui.org/downloads/soapui/>)

Open SoapUI .

Click New SOAP Project



In the popup:

Project Name: CalculatorSOAP (your choice)

Initial WSDL: Paste this URL <http://127.0.0.1:8001/?wsdl>

You will now see in the LEFT panel:

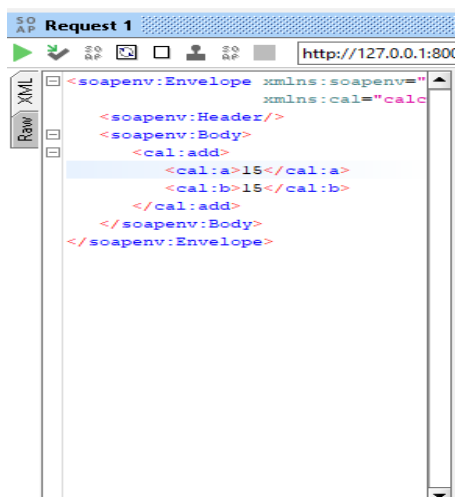
Project: CalculatorSOAP

Binding **add** (this is your SOAP method)

Click **add** → You will see a **sample SOAP Request**.

Enter sample request:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:cal="calculator.soap.service">
  <soapenv:Header/>
  <soapenv:Body>
    <cal:add>
      <cal:a>10</cal:a>
      <cal:b>15</cal:b>
    </cal:add>
  </soapenv:Body>
</soapenv:Envelope>
```



Click Send - Press Green Button

You will now see in the RIGHT panel:

✓You will get the response:

<tns:addResult>25</tns:addResult>

3)GraphQL API

- **GraphQL** is a powerful API query language developed by Facebook that enables precise and efficient data retrieval.
- It allows clients to **request exactly the required data**, eliminating over-fetching and under-fetching issues.
- GraphQL operates through a **single endpoint**, simplifying API management and communication.

Common Use Cases

- Applications requiring customizable data responses
- Frontend-driven systems with varying data needs
- High-performance APIs with optimized network usage

Industry Implementations

- Facebook
- GitHub (GraphQL API)
- Shopify

Creating a Simple GraphQL Web Service

- GraphQL services can be implemented using **Python with the Strawberry framework**.
- Strawberry enables schema-first development with clear type definitions.

Step 1: Install

pip install strawberry-graphql uvicorn

```
D:\cc_lab\week1>pip install strawberry-graphql uvicorn
Collecting strawberry-graphql
  Downloading strawberry_graphql-0.287.0-py3-none-any.whl.metadata (7.7 kB)
Requirement already satisfied: uvicorn in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (0.38.0)
Collecting graphql-core<3.4.0,>=3.2.0 (from strawberry-graphql)
  Downloading graphql_core-3.2.7-py3-none-any.whl.metadata (11 kB)
Collecting lia-web<0.2.1 (from strawberry-graphql)
  Downloading lia_web-0.2.3-py3-none-any.whl.metadata (1.1 kB)
Collecting packaging>=23 (from strawberry-graphql)
  Downloading packaging-25.0-py3-none-any.whl.metadata (3.3 kB)
Collecting python-dateutil<3.0,>=2.7 (from strawberry-graphql)
```

Note: Strawberry GraphQL is a Python library used to build GraphQL APIs

Step 2: Create **graphql_service.py** file inside your week1 folder and write the following code.

```
import strawberry
from fastapi import FastAPI
from strawberry.fastapi import GraphQLRouter
@strawberry.type
class Query:
    @strawberry.field
```

```

def hello(self) -> str:
    return "Hello from GraphQL"
@strawberry.field
def add(self, a: int, b: int) -> int:
    return a + b
schema = strawberry.Schema(query=Query)
app = FastAPI()
graphql_app = GraphQLRouter(schema)
app.include_router(graphql_app, prefix="/graphql")

```

Step 3: Run

uvicorn graphql_service:app --reload

```

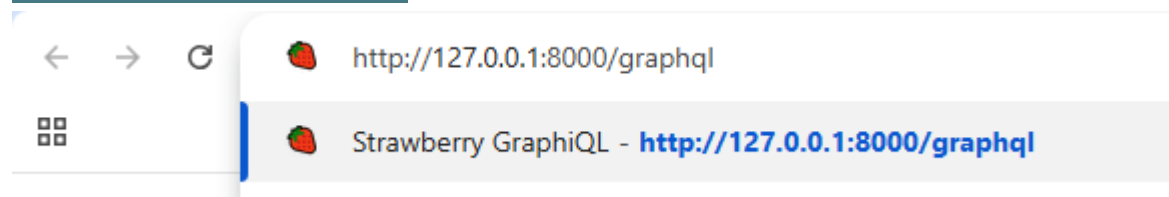
D:\cc_lab\week1>uvicorn graphql_service:app --reload
[32mINFO[0m: Will watch for changes in these directories: ['D:\\cc_lab\\week1']
[32mINFO[0m: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
[32mINFO[0m: Started reloader process [36m[1m8780[0m] using [36m[1mStatReload[0m
[32mINFO[0m: Started server process [36m[1m1964[0m]
[32mINFO[0m: Waiting for application startup.
[32mINFO[0m: Application startup complete.

```

Step 4: Test in GraphQL Playground

Open Browser:

<http://127.0.0.1:8000/graphql>



Run query:

```

{
  add(a: 10, b: 20)
}

```

Press Run symbol in left panel



Output: (In RIGHT panel.)

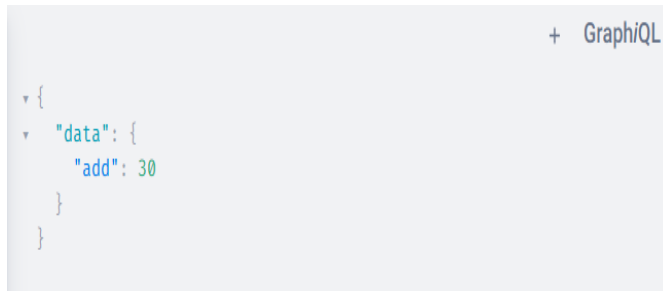
```

{
  "data": {
    "add": 30
  }
}

```



```
}  
}
```



4)gRPC Service

- **gRPC** is a high-performance remote procedure call (RPC) framework developed by Google that uses **Protocol Buffers** for efficient data serialization.
- It enables **low-latency and high-throughput communication**, making it ideal for distributed systems.

Key Applications

- Microservices-based architectures
- High-speed backend-to-backend communication
- Cloud-native and containerized systems

Industry Adoption

- Netflix microservices
- Kubernetes internal service communication

Creating a Simple gRPC Web Service

- gRPC services can be implemented using **Python with protocol buffer definitions**.
- Ultra-fast modern RPC system by Google.

Step 1: Install

pip install grpcio grpcio-tools

```
D:\cc_lab\week1>pip install grpcio grpcio-tools  
Requirement already satisfied: grpcio in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (1.76.0)  
Requirement already satisfied: grpcio-tools in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (1.76.0)  
Requirement already satisfied: typing-extensions~=4.12 in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from grpcio) (4.15.0)  
Requirement already satisfied: protobuf<7.0.0,>=6.31.1 in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from grpcio-tools) (6.33.1)  
Requirement already satisfied: setuptools in c:\users\admin\appdata\local\programs\python\python310\lib\site-packages (from grpcio-tools) (65.5.0)
```

Step 2: Create **calculator.proto file inside you're your week1 folder and write the following code**

and save it.

```
syntax = "proto3";  
service Calculator {  
  rpc Add (AddRequest) returns (AddResponse);  
}  
message AddRequest {  
  int32 a = 1;  
  int32 b = 2;
```

```
}  
message AddResponse {  
  int32 result = 1;  
}
```

Step 3: Generate Python Files to run the following command inside command prompt.

```
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. calculator.proto
```

```
D:\cc_lab\week1>python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. calculator.proto
```

```
D:\cc_lab\week1>
```

Step 4: Create Server (server.py) file inside week1 folder and write the following code and save it.

```
import grpc  
from concurrent import futures  
import calculator_pb2  
import calculator_pb2_grpc  
class CalculatorServicer(calculator_pb2_grpc.CalculatorServicer):  
  def Add(self, request, context):  
    return calculator_pb2.AddResponse(result=request.a + request.b)  
server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))  
calculator_pb2_grpc.add_CalculatorServicer_to_server(CalculatorServicer(), server)  
server.add_insecure_port("[::]:50051")  
server.start()  
server.wait_for_termination()
```

Step 5: Create Client (client.py) file inside week1 folder and write the following code and save it.

```
import grpc  
import calculator_pb2  
import calculator_pb2_grpc  
channel = grpc.insecure_channel("localhost:50051")  
stub = calculator_pb2_grpc.CalculatorStub(channel)  
response = stub.Add(calculator_pb2.AddRequest(a=10, b=20))  
print("Result:", response.result)
```

Step 6: Run

Terminal 1: run the server by following command

```
python server.py
```

Terminal 2: run the client by following command

```
python client.py
```

Output:

```
Result:30
```

